

HOMELAB PORTAL

PUBLICATION ARCHITECTURE

STATIC SITE // PUBLIC DROP // STATUS



Git-managed portal + controlled AI file publishing

Caddy + Hugo + git-sync + Skills Runner

IMPLEMENTATION BRIEF / AUG 2026



80s CYBERPUNK x BRICK-BUILT ARCADE



Replace Ghost with a static, Git-managed homelab portal and a separate public drop zone. The portal is curated and versioned; ad-hoc files are immediately publishable without requiring a Git commit or site rebuild.

RECOMMENDED STACK: Caddy as the static origin server, Hugo for authored pages and thoughts, git-sync for atomic pull-based deployment, and a dedicated /data/media/public mount that Skills Runner can write to while Caddy sees it read-only.

CURATED PORTAL

Landing page, arcade, projects, thoughts and design assets live in Git. Changes publish through a normal commit/push workflow.

STATUS PANEL

Homepage reads a small sanitized status.json generated by a separate status publisher. Do not expose the Docker socket or internal host details to the web container.

PUBLIC DROP ZONE

AI-generated PDFs, images, audio, video and other shareable files land in /data/media/public and appear immediately under a themed /files/ browser.

SIMPLE OPERATIONS

No CMS database, login/admin UI, plugin ecosystem or dynamic application runtime is required for normal page delivery.

Why this fits the homelab

- A Git push becomes the publishing workflow for permanent content.
- A file copy becomes the publishing workflow for temporary or generated content.
- Static serving greatly reduces maintenance and public attack surface compared with a CMS.
- The architecture creates a clean capability that agents can call: publish a safe file and receive its public URL.
- The visual experience can feel like a personal arcade/lab console instead of a traditional family blog.

CORE RULE: Only the explicit public directory is Internet-visible. Never mount the full media tree, skills workspace, home directories, model storage, or application data into the public web container.

The public site should feel like a retro-futurist home base: neon terminal typography, dark cyberpunk surfaces, colorful brick-built modules, subtle arcade motion, and useful live-ish status without leaking internal topology.

Route	Purpose	Publishing model
/	Portal landing page + homelab status	Git / Hugo
/arcade/	One-page HTML games and future arcade projects	Git / static assets
/thoughts/	Occasional Markdown posts / notes	Git / Hugo
/lab/	Projects, experiments, interesting builds	Git / Hugo
/files/	Themed browseable public file index	Shared public mount
/files/ai/	Agent/skill generated artifacts	Skills Runner writes

Visual direction

- Base: near-black/navy surfaces with cyan, magenta, violet and amber accent lighting.
- Structure: card modules that resemble chunky interlocking bricks without copying any brand marks.
- Typography: squared/monospace display type for labels; highly readable sans-serif for body text.
- Motion: restrained scan-line, glow, hover and arcade-button effects; honor prefers-reduced-motion.
- Navigation labels can read like console modules: ARCADE, LAB, THOUGHTS, FILES, SYSTEM STATUS.
- Responsive first: the landing page should work cleanly on phones as well as desktop browsers.

DESIGN CONSTRAINT: Do not let the theme compromise readability. Use neon as accent color, not as body-copy color; meet accessible contrast; use semantic HTML, keyboard navigation, visible focus states and alt text.

Suggested homepage modules

SYSTEM STATUS

A few friendly service tiles, overall state, last update and portal version.

LATEST DROP

Most recent public files or a link into /files/.

ARCADE

Large visual tile linking to the existing one-page HTML arcade.

RECENT THOUGHTS

Two or three newest short posts, not a blog-first layout.



Container responsibilities

Component	Responsibility	Write access
Caddy	Serve generated site, custom /files/ listing, headers/compression	None to content; read-only mounts
git-sync	Pull deploy revision and atomically switch current worktree	Only its deployment volume
Hugo build	Convert source content/assets to static site	CI workspace or isolated builder
status publisher	Produce sanitized status JSON	Only runtime status directory
Skills Runner	Create/copy approved public artifacts	Only /data/media/public for publishing

REVERSE PROXY NOTE: If the homelab already has a public reverse proxy, keep it. Run this Caddy origin on an internal port such as 8080 and route the public hostname to it. Do not create a second TLS ingress unless there is a reason to replace the existing edge.

```

/homelab
|-- apps/
|   |-- public-site/
|   |   |-- compose.yaml
|   |   |-- Caddyfile
|   |   |-- browse-template.html
|   |   |-- env.example
|   |
|   |-- data/
|   |   |-- public-site/
|   |   |   |-- git/          # git-sync worktrees + current symlink
|   |   |   |-- runtime/
|   |   |   |   |-- status.json # sanitized, generated data
|   |   |
|   |   |-- media/
|   |   |   |-- public/      # ONLY this media subtree is public
|   |   |   |   |-- ai/
|   |   |   |   |-- files/
|   |   |   |   |-- images/
|   |   |   |   |-- audio/
|   |   |   |   |-- video/

```

Mount policy

Consumer	Host path	Container path	Mode
Caddy	/homelab/data/public-site/git	/srv/git	ro
Caddy	/homelab/data/public-site/runtime	/srv/runtime	ro
Caddy	/homelab/data/media/public	/srv/files	ro
git-sync	/homelab/data/public-site/git	/git	rw
status publisher	/homelab/data/public-site/runtime	/status	rw
Skills Runner	/homelab/data/media/public	/public	rw

Non-negotiable controls

- Do not rely on Caddy hide rules as the security boundary; the filesystem mount itself is the boundary.
- Prevent symlinks in /data/media/public. Caddy documents that a file root is not a sandbox against symlinks that point outside the root.
- Run containers as non-root where practical and set explicit UID/GID ownership for shared mounts.
- Do not mount /var/run/docker.sock into Caddy. The public site should never gain Docker control just to display status.
- Keep Git credentials read-only and scoped to deployment. Prefer a deploy key over a broad personal token for a private repo.
- Treat the public drop zone as untrusted display content even when it is generated by your own AI.

ACTIVE-CONTENT RULE: Curated HTML/JavaScript belongs in Git. For the AI/file drop, block or force-download active web types such as .html, .htm and .js. If you later want AI-generated live HTML demos, serve them from a separate sandbox hostname/origin with a restrictive Content-Security-Policy.

```
homelab-portal/
|-- hugo.toml
|-- content/
|   |-- thoughts/
|   |-- lab/
|-- layouts/
|   |-- _default/
|   |-- partials/
|   |-- index.html
|-- assets/
|   |-- css/
|   |-- js/
|-- static/
|   |-- arcade/      # copy your existing one-page arcade here
|   |-- images/
|-- deploy/          # optional scripts / CI definitions
`-- README.md
```

Recommended deployment model

1. Edit content or code on a workstation or through an AI coding workflow.
2. Commit and push the source branch (usually main).
3. Build Hugo in CI or a controlled build job, not in the public Caddy container.
4. Publish the generated static output to a dedicated deploy branch/repository.
5. git-sync watches that deploy target and atomically updates its current symlink.
6. Caddy serves only the current generated worktree read-only.

WHY A DEPLOY BRANCH: It keeps the public origin extremely dumb: it receives already-built HTML/CSS/JS and only serves it. Source dependencies and build tooling do not need to exist in the Internet-facing container.

Fallback if CI is unavailable

Use a small isolated Hugo builder sidecar that watches the git-sync revision and builds into a separate generated-output directory. This is acceptable, but it adds moving parts. Start with CI if the existing Git host already supports actions/workflows.

Caddy supports static file serving, automatic directory browsing, a custom browse template, JSON/plain-text directory listings by Accept header, and response compression. The custom browse template is the key to making /files/ look like the rest of the portal instead of a generic server index.

```
:8080 {
    encode zstd gzip

    # Public file drop. Root is mounted read-only.
    handle_path /files/* {
        root * /srv/files
        file_server browse /etc/caddy/browse-template.html {
            sort namedirfirst asc
        }
    }

    # Sanitized status output.
    handle_path /status/* {
        root * /srv/runtime
        file_server
    }

    # Generated Hugo site from git-sync's atomic symlink.
    handle {
        root * /srv/git/current
        file_server
    }
}
```

IMPLEMENTATION NOTE: Treat the snippet as a target shape, not copy/paste gospel. The local AI should validate the exact Caddy version, current directive syntax, mount paths and existing reverse-proxy conventions before applying it.

Browse-template experience

- Use the same CSS variables, typography and navigation partial as the main portal.
- Display breadcrumb navigation, directories first, filename, modified time and human-readable size.
- Add a short PUBLIC DROP explanatory banner and a link back to the portal home.
- Add file-type icons with simple CSS/inline SVG only; no external CDN is required.
- Keep the listing useful without JavaScript. Progressive enhancement is optional.
- Do not reveal filesystem paths, internal mount names, uploader identity or service metadata.

The homepage status module should be informative but deliberately boring from a security perspective. It is a presentation layer, not an operations console.

```
GET /status/status.json

{
  "overall": "online",
  "updated_at": "2026-08-24T15:00:00Z",
  "portal_revision": "a1b2c3d",
  "services": [
    {"name": "AI Lab", "state": "online"},
    {"name": "Media", "state": "online"},
    {"name": "Automation", "state": "degraded"},
    {"name": "Public Files", "state": "online"}
  ]
}
```

Safe implementation pattern

1. A small status publisher runs on a schedule (for example every 30-60 seconds).
2. It performs only the checks you explicitly configure: HTTP health endpoints, ICMP if appropriate, or queries to an existing monitoring API.
3. It maps those checks to friendly public names and coarse states such as online, degraded or offline.
4. It writes status.json atomically into /data/public-site/runtime.
5. Caddy serves that file read-only; the homepage fetches it and renders the tiles.

DO NOT EXPOSE: Internal IP addresses, hostnames, container names, versions, ports, CPU/RAM telemetry, private URLs, Docker events, stack traces, tokens, environment variables or raw monitoring payloads.

Optional phase-two integrations

- Read from an existing Uptime Kuma/Prometheus/status API rather than duplicating checks.
- Show a small 24-hour availability sparkline only if the source can provide sanitized aggregate data.
- Show portal Git revision and deploy time for easy troubleshooting.
- Show public-drop file count and total published size if that is useful and not sensitive.

Give Skills Runner a narrow publishing capability instead of generic write access to arbitrary homelab storage. The skill should encapsulate validation, copying, naming and URL construction.

```
publish_file(
    source_path,          # must be inside approved source roots
    destination_name=None,
    subdirectory="ai",
    overwrite=False
) -> {
    "path": "/public/ai/example.pdf",
    "url": "https://portal.example.com/files/ai/example.pdf",
    "size_bytes": 123456,
    "sha256": "...
}
```

Skill behavior

- Normalize and validate the requested destination. Reject absolute paths, .. traversal, null bytes and unexpected path separators.
- Restrict source files to configured roots rather than allowing the agent to copy any file on the host.
- Create directories with predictable permissions; publish files with world-readable but non-executable permissions.
- Write to a temporary filename in the destination directory and rename atomically after a successful copy.
- Generate collision-safe names when overwrite=false and return the final public URL.
- Optionally calculate SHA-256 and size for the response/log.
- Reject symlinks and special files. Only publish normal files.
- Block or force a safe extension policy for active web content. HTML/JS belongs in the Git workflow.
- Log publish/unpublish operations locally without exposing those logs publicly.

USEFUL FOLLOW-ON SKILLS: add `publish_directory` for passive bundles, `list_public_files` for agent discovery, and `unpublish_file` that moves content into a private trash/quarantine directory rather than immediately deleting it.

The local AI should adapt the following service shape to the existing Docker conventions, networks, UID/GID scheme, reverse proxy labels and secrets management. Avoid replacing the homelab edge or network design unless necessary.

```
services:
  portal:
    image: caddy:<PINNED_VERSION>
    restart: unless-stopped
    read_only: true
    tmpfs:
      - /config
      - /data
    ports:
      - "127.0.0.1:<ORIGIN_PORT>:8080" # or internal network only
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - ./browse-template.html:/etc/caddy/browse-template.html:ro
      - /homelab/data/public-site/git:/srv/git:ro
      - /homelab/data/public-site/runtime:/srv/runtime:ro
      - /homelab/data/media/public:/srv/files:ro
    networks: [public_edge]

  git-sync:
    image: registry.k8s.io/git-sync/git-sync:<PINNED_V4>
    restart: unless-stopped
    environment:
      GITSYNC_REPO: <DEPLOY_REPO>
      GITSYNC_REF: <DEPLOY_BRANCH>
      GITSYNC_ROOT: /git
      GITSYNC_LINK: current
      GITSYNC_PERIOD: 30s
    volumes:
      - /homelab/data/public-site/git:/git

  status-publisher:
    # tiny local script/container; no public ingress
    volumes:
      - /homelab/data/public-site/runtime:/status
    networks: [monitoring]
```

GIT-SYNC V4: Current git-sync v4 documentation uses `--link` rather than the old `--dest` flag and `--period` rather than `--wait`. It publishes changes via a worktree + symlink so consumers do not see a partially updated checkout.

Operational hardening

- Pin container versions after validation; do not deploy latest tags.
- Add health checks, restart policies and resource limits appropriate to the host.
- Keep the Caddy filesystem read-only except for any runtime locations Caddy itself explicitly requires.
- Use a dedicated Docker network from the edge proxy to the portal origin.
- Back up the Git repo normally; public-drop data follows the media/public backup policy.
- Document rollback: revert Git for the portal; remove/move a file for the public drop.

Implement this as a controlled migration, not a big-bang replacement. Keep Ghost available until the new portal passes local and external tests, then cut the reverse proxy over.

01 INVENTORY AND PRESERVE

Record the current Ghost hostname, edge proxy route, Docker network, ports, volumes, certificates, DNS, Git hosting, user/group IDs and Skills Runner mounts. Back up the Ghost compose/config before changing anything.

02 CREATE PUBLIC BOUNDARIES

Create /data/public-site/{git,runtime} and /data/media/public with ownership/permissions that support git-sync, status-publisher and Skills Runner while keeping Caddy read-only.

03 BOOTSTRAP THE PORTAL REPO

Create Hugo source, custom theme/layouts, responsive landing page and placeholder modules. Copy the existing arcade into static/arcade without rewriting it unless required.

04 BUILD THE 80S VISUAL SYSTEM

Implement CSS tokens, brick-built cards, neon borders, accessible focus/contrast, reduced-motion handling and reusable header/footer/navigation components.

05 IMPLEMENT /FILES/

Export/inspect Caddy browse template, create a custom themed version, mount the public directory read-only, then verify nested directories and multiple file types.

06 IMPLEMENT DEPLOYMENT

Build source to a deploy branch/repo, configure git-sync v4 with a deploy credential if needed, and verify atomic revision changes without restarting Caddy.

07 IMPLEMENT PUBLIC STATUS

Create the sanitized status publisher and homepage fetch/render logic. Verify that the JSON contains only intended public data.

08 UPDATE SKILLS RUNNER

Add the /public mount and publish_file capability with validation, atomic write, permissions, active-content policy and URL response.

09 SECURITY TEST

Test traversal, symlinks, hidden files, dotfiles, active extensions, oversized names, path collisions, invalid inputs and direct requests for files outside the public root.

10 CUT OVER AND OBSERVE

Switch the existing reverse proxy hostname from Ghost to the portal origin, test externally, watch logs/health, then stop Ghost. Remove Ghost only after the rollback window is comfortable.

Functional acceptance

- Portal home loads from the public hostname and all major sections work on desktop and mobile.
- Existing one-page arcade runs under /arcade/ with its assets intact.
- A Markdown thought committed to Git appears after the deploy sync without manually touching the web server.
- A PDF copied through Skills Runner becomes visible under /files/ and the skill returns the correct URL.
- /files/ listing matches the portal visual language and works without JavaScript.
- Status tiles update from sanitized status.json and degrade gracefully if the file is stale/unavailable.
- Caddy has read-only access to site content, runtime status and public files.

Security acceptance

- Attempts to request ../ or encoded traversal cannot escape the public roots.
- Symlink publishing is rejected and a manually-created symlink cannot expose data outside the intended root.
- Skills Runner cannot publish arbitrary host paths outside approved source roots.
- Active file types in the drop zone follow the decided block/download policy.
- No Docker socket, credentials, environment dumps, private monitoring payloads, internal addresses or hostnames appear in public responses.
- The portal origin exposes only the expected port/network path from the existing edge.

Operations acceptance

- git-sync recovers from a temporary Git outage and resumes without manual intervention.
- A bad portal change can be rolled back with a Git revert or deploy-ref change.
- Public files can be removed without modifying the site repository.
- Logs are available locally for Caddy, git-sync, status publishing and publish_file operations.
- The migration notes document how to restore the old Ghost route if necessary.

SUCCESS STATE: After cutover, the portal should be boring to operate: Git push for curated changes, publish_file for ad-hoc artifacts, and normal Docker health/logs for everything else.

The following instructions are intentionally written for a capable local coding agent working inside an existing homelab. They assume a 128K configured context window, which is ample for the repo/config inspection and staged implementation below. The official Qwen3.8-27B model card currently lists a larger native context window, but the implementation should respect the local runtime limit you actually configured.

AGENT OPERATING MODE: Inspect first, preserve existing conventions, make small reversible commits, and validate each boundary before moving to the next phase. Do not redesign unrelated infrastructure just because a different pattern is possible.

Context package to provide the local AI

- This PDF.
- Current homelab Docker Compose files for Ghost, Skills Runner and the public reverse proxy.
- Current directory tree for the relevant apps/data paths (redact secrets).
- Reverse-proxy config/labels for the current Ghost hostname.
- Skills Runner source/config plus its existing skill conventions.
- Git host details (GitHub, Forgejo, Gitea, etc.) and whether CI/actions are available.
- The existing one-page arcade directory/files.
- Any existing status/monitoring service that could be used as a read-only source.

Required agent constraints

- Never print, commit, or copy secrets into generated examples or logs.
- Do not remove Ghost until the new portal has passed external validation and rollback is documented.
- Do not expose any new port publicly if the existing reverse proxy can route to an internal origin.
- Do not mount parent media directories into Caddy or the publishing skill.
- Do not give the public web container the Docker socket.
- Do not permit AI-dropped HTML/JS to execute on the main portal origin.
- Ask for a human decision only when a choice changes credentials, DNS, destructive deletion, or the trust model; otherwise choose the safest reversible default and continue.

ROLE

You are the implementation engineer for an existing Docker homelab. Your job is to replace Ghost at the current public homelab hostname with a static portal while preserving the existing edge proxy, network conventions and data layout wherever practical.

PRIMARY OUTCOME

Build a cyberpunk/retro-arcade portal using Hugo + Caddy + git-sync, add a themed /files/ index backed by a narrow public shared directory, add a sanitized status panel, and extend Skills Runner with a safe publish_file capability that returns a public URL.

WORKING RULES

1. Inspect before editing. Do not assume paths, proxy software, Git host, UID/GID, or ports.
2. Never reveal or commit secrets. Use .env.example placeholders and existing secret patterns.
3. Work in small reversible commits. Keep Ghost operational until cutover validation passes.
4. Prefer existing homelab conventions over introducing new frameworks.
5. Caddy content mounts must be read-only. No Docker socket in the portal container.
6. Only /data/media/public (or the equivalent confirmed path) is public media storage.
7. Treat dropped active web content as unsafe on the main origin; block/force-download it.
8. Run tests after every phase and record results in IMPLEMENTATION_NOTES.md.

PHASE 0 - DISCOVERY

- Inventory current Ghost compose/config, edge proxy route, Docker networks, hostname, ports, volume roots, UID/GID and health-check conventions.
- Inventory Skills Runner architecture: skill registration, filesystem mounts, validation patterns, response schema and test framework.
- Determine Git remote type and whether CI/actions can build Hugo to a deploy branch.
- Locate the arcade files and inspect relative asset paths.
- Identify existing monitoring/status data sources.
- Produce a short discovered-state summary and a proposed file-change list before edits.

PHASE 1 - FILESYSTEM + REPO SCAFFOLD

- Create confirmed equivalents of data/public-site/git, data/public-site/runtime and data/media/public with least-privilege ownership.
- Scaffold homelab-portal Hugo repo with content/, layouts/, assets/, static/arcade/.
- Copy arcade unchanged first; only fix paths if tests prove they break.
- Add README with local build, deploy, rollback and content-authoring instructions.

PHASE 2 - PORTAL UI

- Build responsive landing page with modules: SYSTEM STATUS, ARCADE, LAB, THOUGHTS, FILES.
- Implement 80s cyberpunk + brick-built arcade design using local CSS/assets.
- Use accessible contrast, semantic markup, keyboard focus and prefers-reduced-motion.
- Avoid external runtime CDN dependencies unless already standard in the homelab.

PHASE 3 - CADDY + /FILES/

- Add Caddy origin config on internal-only port/network consistent with existing proxy.
- Serve generated portal from git-sync current worktree read-only.
- Serve /files/ from the public media mount read-only with a custom browse template matching the portal styles; directories first; useful filename/size/time; no filesystem leakage.
- Serve sanitized /status/status.json from a separate read-only runtime mount.
- Add security headers appropriate for a static site and implement the active-content policy.

PHASE 4 - GIT DEPLOYMENT

- Prefer CI/action build: source branch -> Hugo build -> deploy branch/repo.
- Configure git-sync v4 against deploy target using current --link/--period semantics.
- Use a read-only deploy key/credential if the repo is private.
- Verify a Git push changes the portal without restarting Caddy and without a partial update.

PHASE 5 - STATUS PUBLISHER

- Build the smallest reasonable status publisher for the existing environment.
- Prefer an existing monitoring API if one is available; otherwise perform explicitly configured health checks only.
- Write status.json atomically every 30-60 seconds with friendly names and coarse states.
- Public payload MUST NOT include internal IPs/hostnames, versions, ports, credentials, stack traces, environment data, Docker details or raw monitoring payloads.
- Frontend must handle stale/missing JSON gracefully and show last-updated time.

PHASE 6 - SKILLS RUNNER PUBLISH_FILE

- Add only the confirmed public directory as a RW mount, e.g. /data/media/public:/public.
- Implement publish_file(source_path, destination_name?, subdirectory?, overwrite=false).
- Restrict allowed source roots; reject symlinks/special files/path traversal/absolute dests.
- Copy to a temp file then rename atomically; set sane file/dir modes.
- Generate a collision-safe filename when overwrite=false.
- Block or force-download active web types; do not use this skill to deploy HTML/JS apps.
- Return final public URL, path, byte size and SHA-256. Log operation locally.
- Add unit tests for validation and an integration test that publishes a sample PDF/image.

PHASE 7 - SECURITY + FUNCTIONAL TEST MATRIX

Test at minimum:

- /, /arcade/, /thoughts/, /lab/, /files/, nested /files/ directories.
- Git update appears atomically through git-sync.
- public file appears immediately without a site rebuild.
- invalid paths: ../, encoded traversal, absolute paths, dotfiles, long names, collisions.
- symlink file and symlink directory attempts.
- blocked/forced-download active content behavior.
- direct requests for known private sibling paths fail.
- portal container cannot write Git/status/public-file mounts.
- no container has more network or filesystem access than required.
- mobile viewport, keyboard navigation, reduced-motion mode, missing status JSON.

PHASE 8 - CUTOVER + CLEANUP

- Capture a pre-cutover rollback procedure.
- Change only the existing edge route needed to point the hostname at the new portal origin.
- Validate externally over HTTPS from outside the LAN.
- Confirm logs, cache behavior, directory browsing, arcade, status and a published test file.
- Stop Ghost but do not delete its data immediately.
- After the chosen rollback window, remove unused Ghost runtime resources while retaining the backup/export per the existing homelab backup policy.

FINAL DELIVERABLES

- Working homelab-portal Git repo.
- Updated public-site compose/Caddy configuration.
- Custom Caddy browse-template styled with the portal.
- Status publisher + documented sanitized schema.
- Skills Runner publish_file implementation + tests.
- IMPLEMENTATION_NOTES.md with discovered assumptions, changes, tests and rollback steps.
- README.md with normal operations: publish page, publish file, remove file, rollback portal.

DEFINITION OF DONE

Do not call the migration complete until every acceptance item in the accompanying PDF has been demonstrated or explicitly documented as deferred with a reason.

Architecture decisions

Decision	Default
Portal framework	Hugo-generated static site
Origin server	Caddy behind existing edge proxy
Curated deploy	Git push -> build -> deploy target -> git-sync
Ad-hoc publishing	/data/media/public, Caddy RO / Skills Runner RW
File index	Caddy file_server browse with custom template
Status	Sanitized JSON from separate status publisher
Active generated HTML/JS	Not executable on main portal origin
Migration	Parallel validation, then proxy cutover, Ghost retained for rollback window

Current implementation notes verified against upstream docs

- Caddy file_server supports browse mode and a custom browse template. Browse output can also be requested as JSON or plain text.
- Caddy cautions that the site root is not a sandbox against symlinks pointing outside the root; therefore the mount boundary and symlink policy are explicit parts of this design.
- git-sync v4 periodically synchronizes a Git repo and publishes each revision through a worktree + symlink for atomic consumption. Current v4 naming uses --link and --period.
- Hugo remains a static site generator with a straightforward Git-oriented content workflow.
- The official Qwen3.8-27B model card lists 27B parameters and a 262,144-token native context window; this handoff intentionally assumes the user-configured 128K runtime context.

References

Caddy file_server: https://caddyserver.com/docs/caddyfile/directives/file_server

Caddy templates: <https://caddyserver.com/docs/caddyfile/directives/templates>

Hugo quick start: <https://gohugo.io/getting-started/quick-start/>

git-sync v4: <https://github.com/kubernetes/git-sync>

git-sync serving example: <https://github.com/kubernetes/git-sync/blob/master/docs/kubernetes.md>

Qwen3.8-27B model card: <https://huggingface.co/Qwen/Qwen3.8-27B>

END STATE: A public homelab site that feels personal and fun, but operates like infrastructure: curated pages are versioned, generated files are narrowly published, status is sanitized, and the Internet-facing server has almost nothing it is allowed to change.